Sorting IV - Bucket Sort and Radix Sort

Shahriar Ivan
Lecturer, Department of CSE, IUT

Supporting Example for Bucket Sort

Suppose we are sorting a large number of local phone numbers, for example, all residential phone numbers in the 416 area code region (approximately four million)

We could use quick sort, however that would require an array which is kept entirely in memory

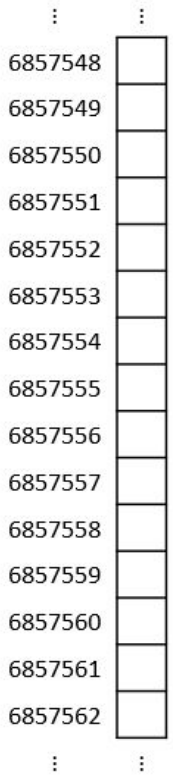
Supporting Example for Bucket Sort

Instead, consider the following scheme:

- Create a bit vector with 10 000 000 bits
  - This requires $10^7/1024/1024/8 \approx 1.2$ MiB
- Set each bit to 0 (indicating false)
- For each phone number, set the bit indexed by the phone number to 1 (true)
- Once each phone number has been checked, walk through the array and for each bit which is 1, record that number

Supporting Example for Bucket Sort

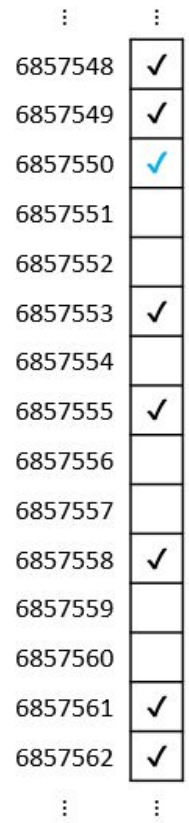
For example, consider this section within the bit array



Supporting Example for Bucket Sort

For each phone number, set the corresponding bit

For example, 685-7550 is a residential phone number



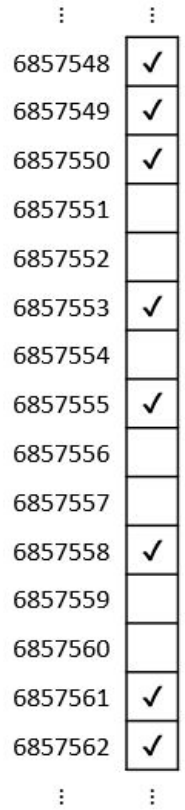
Supporting Example for Bucket Sort

At the end, we just take all the numbers out that were checked:

…, 685-7548, 685-7549, 685-7550,

685-7553, 685-7555, 685-7558,

685-7561, 685-5762, …



Supporting Example for Bucket Sort

In this example, the number of phone numbers (4 000 000) is comparable to the size of the array (10 000 000)

The run time of such an algorithm is $\Theta(n)$:

- we make one pass through the data,
- we make one pass through the array and extract the phone numbers which are true

The Algorithm

This approach uses very little memory and allows the entire structure to be kept in main memory at all times

We will term each entry in the bit vector a bucket

We fill each bucket as appropriate

Another Example

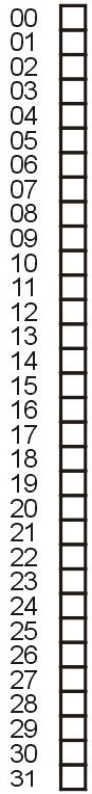
Consider sorting the following set of unique integers in the range 0, ..., 31:

20 1 31 8 29 28 11 14 6 16 15

27 10 4 23 7 19 18 0 26 12 22

Create an bit-vector with 32 buckets

This requires 4 bytes



Another Example

For each number, set the corresponding bucket to 1

Now, just traverse the list and record only those numbers for which the bit is 1 (true):

0 1 4 6 7 8 10 11 12 14 15

16 18 19 20 22 23 26 27 28 29 31

Runtime Analysis

How is this so fast?

An algorithm which can sort arbitrary data must be $\Omega(n \ln(n))$

But in this case, we don't have arbitrary data: we have one further constraint, that the items being sorted fall within a certain range

So, using this assumption, we can reduce the run time

Modification

Modification: what if there are repetitions in the data

- In this case, a bit vector is insufficient

Two options, each bucket is either:

- a counter, or
- a linked list

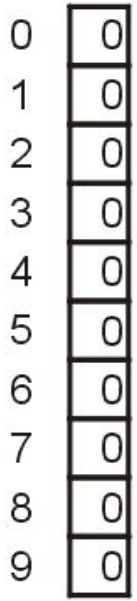
The first is better if objects in the bin are the same

Example with duplicate values

Sort the digits

0 3 2 8 5 3 7 5 3 2 8 2 3 5 1 3 2 8 5 3 4 9 2 3 5 1 0 9 3 5 2 3 5 4 2 1 3

We start with an array of 10 counters, each initially set to zero:



Example with duplicate values

Moving through the first 10 digits

0 3 2 8 5 3 7 5 3 2 8 2 3 5 1 3 2 8 5 3 4 9 2 3 5 1 0 9 3 5 2 3 5 4 2 1 3

we increment the corresponding buckets

Example with duplicate values

Moving through remaining digits

0 3 2 8 5 3 7 5 3 2 8 2 3 5 1 3 2 8 5 3 4 9 2 3 5 1 0 9 3 5 2 3 5 4 2 1 3

we continue incrementing the corresponding buckets

Example with duplicate values

We now simply read off the number of each occurrence:

0 0 1 1 1 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 4 4 5 5 5 5 5 5 7 8 8 8 9 9

For example

- There are seven 2s
- There are two 4s

Radix Sort

Suppose we want to sort 10 digit numbers with repetitions

- We could use bucket sort, but this would require the use of 1010 buckets
- With one byte per counter, this would require 9 GiB

This may not be very practical…

Radix Sort

Consider the following scheme

Given the numbers

16 31 99 59 27 90 10 26 21 60 18 57 17

If we first sort the numbers based on their last digit only, we get:

90 10 60 31 21 16 26 27 57 17 18 99 59

Now sort according to the first digit:

10 16 17 18 21 26 27 31 57 59 60 90 99

Radix Sort

The resulting sequence of numbers is a sorted list

Thus, consider the following algorithm:

- Suppose we are sorting decimal numbers
- Create an array of 10 queues
- For each digit, starting with the least significant
    - Place the ith number in to the bin corresponding with the current digit
    - Remove all digits in the order they were placed into the bins in the order of the bins

Radix Sort

Suppose that two n-digit numbers are equal for the first m digits:

$$a = a_n a_{n-1} a_{n-2} \cdots a_{n-m+1} \mathbf{a_{n-m}} \cdots a_1 a_0$$

$$b = a_n a_{n-1} a_{n-2} \cdots a_{n-m+1} \mathbf{b_{n-m}} \cdots b_1 b_0$$

where $\mathbf{a_{n-m} < b_{n-m}}$

For example, 103574 < 103892 because 1 = 1, 0 = 0, 3 = 3 but 5 < 8

Then, on iteration n – m, a will be placed in a lower bin than b

When they are taken out, a will precede b in the list

Example 1

Sort the following decimal numbers:

86 198 466 709 973 981 374 766 473 342

First, interpret 86 as 086

Example 1

Next, create an array of 10 queues:

0				
1				
2				
3				
4				
5				
6				
7				
8				
9				

Example 1

Push according to the 3rd digit:

086 198 466 709 973 981 374 766 473 342

0				
1	981			
2	342			
3	973	473		
4	374			
5				
6	086	466	766	
7				
8	198			
9	709			

and dequeue: **981 342 973 473 374 086 466 766 198 709**

Example 1

Enqueue according to the 2nd digit:

981 342 973 473 374 086 466 766 198 709

0	709			
1				
2				
3				
4	342			
5				
6	466	766		
7	973	473	374	
8	981	086		
9	198			

and dequeue: 709 342 466 766 973 473 374 981 086 198

Example 1

Enqueue according to the 1st digit:

709 342 466 766 973 473 374 981 086 198

0	086			
1	198			
2				
3	342	374		
4	466	473		
5				
6				
7	709	766		
8				
9	973	981		

and dequeue: **086 198 342 374 466 473 709 766 973 981**

Example 1

The numbers

086 198 342 374 466 473 709 766 973 981

are now in order

The next example uses the binary representation of numbers, which is even easier to follow

Example 2

Sort the following base 2 numbers:

1111 11011 11001 10000 11010 101 11100 111 1011 10101

First, interpret each as a 5-bit number:

01111 11011 11001 10000 11010 00101 11100 00111 01011 10101

Next, create an array of two queues:

0								
1								

Example 2

Place the numbers

01111 11011 11001 10000 11010 00101 11100 00111 01011 10101

into the queues based on the 5th bit:

0	10000	11010	11100					
1	01111	11011	11001	00101	00111	01011	10101	

Remove them in order:

10000 11010 11100 01111 11011 11001 00101 00111 01011 10101

Example 2

Place the numbers

10000 11010 11100 01111 11011 11001 00101 00111 01011 10101

into the queues based on the 4th bit:

0	10000	11100	11001	00101	10101			
1	11010	01111	11011	00111	01011			

Remove them in order:

10000 11100 11001 00101 10101 11010 01111 11011 00111 01011

Example 2

Place the numbers

10000 11100 11001 00101 10101 11010 01111 11011 00111 01011

into the queues based on the 3rd bit:

0	10000	11001	11010	11011	01011			
1	11100	00101	10101	01111	00111			

Remove them in order:

10000 11001 11010 11011 01011 11100 00101 10101 01111 00111

Example 2

Place the numbers

10000 11001 11010 11011 01011 11100 00101 10101 01111 00111

into the queues based on the 2nd bit:

0	10000	00101	10101	00111				
1	11001	11010	11011	01011	11100	01111		

Remove them in order:

10000 00101 10101 00111 11001 11010 11011 01011 11100 01111

Example 2

Place the numbers

10000 00101 10101 00111 11001 11010 11011 01011 11100 01111

into the queues based on the 1st bit:

0	00101	00111	01011	01111				
1	10000	10101	11001	11010	11011	11100		

Remove them in order:

00101 00111 01011 01111 10000 10101 11001 11010 11011 11100

Example 2

The numbers

00101 00111 01011 01111 10000 10101 11001 11010 11011 11100

are now in order

This required 5n enqueues and dequeues

In this case, it n = 10

Run-time Analysis

Bucket sort is $\Theta(n + m)$ where m is the number of buckets

- Restricting ourselves to either decimal digits or bits ensures that $m = \Theta(1)$

How often must we iterate to sort numbers on the range 0, …, m – 1?

- We require $\lfloor \log_{10}(m) \rfloor + 1$ digits or $\lfloor \log_2(m) \rfloor + 1$ bits
- This is why Arabic numbers are so powerful

Run time is therefore $\Theta(n \ln(m))$

- For this to be faster than previous sorting algorithms, it must be true that $\ln(m) < \ln(n)$ or $m << n$
- Therefore, it is only truly useful if we are sorting lists of relatively small range of numbers with very significant amount of duplications

End of Lecture